CSCI 5551: Autonomous Drone 3D Mapping with Stereo SLAM

Vamsi Krishna Kocherla, Vivek Kumar, Greta Brown, Prapti Bordoloi

koche156@umn.edu, kuma0355@umn.edu, brow6802@umn.edu, bordo025@umn.edu

Problem Description and Motivation

- Need for autonomous indoor drone surveillance
- GPS can be unreliable in indoor areas causing localization and navigation to be difficult
- Precise navigation and obstacle avoidance necessary for navigating through dynamic and/or cluttered spaces
- Stereo SLAM enables depth perception and detailed mapping for drone navigation

Problem Description and Approach

- Design a robust indoor 3D mapping solution for drones
- Utilize stereo cameras to achieve accurate and reliable depth estimation for obstacle detection
- Leverage Isaac ROS Depth Segmentation for precise and real-time obstacle mapping
- Combine PX4 flight control with advanced perception tools

Related Work

PX4 Integration for Autonomous Navigation

- H. D. K. Motlagh et al. (2019): Utilized PX4 for drone position estimation combined with Visual SLAM and IMU in GPS-denied environments.
- McGuire et al. (2017): Explored velocity estimation and obstacle avoidance using PX4 and stereo vision for autonomous drones.

Stereo Camera for Depth Perception

- Xue and Lv (2022): Fused point and line features for Stereo Visual SLAM, enhancing depth perception in complex environments.
- Mur-Artal and Tardós (2017): ORB-SLAM2 for stereo vision, enabling robust SLAM with accurate depth information.
- Jin et al. (2018): Used stereo cameras for indoor positioning, emphasizing localization accuracy.

Related Work

Bi3D Proximity Segmentation

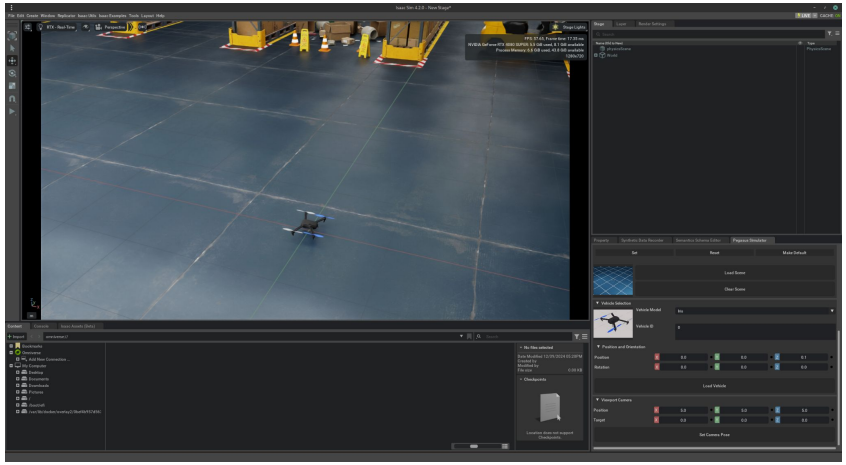
- Wang et al. (2023): Proposed LF-VISLAM utilizing segmentation and large field-of-view cameras, which aligns conceptually with proximity-based segmentation.

Isaac ROS Depth Segmentation

- Duba et al. (2024): Focused on stereo vision-based object detection, providing insights into depth segmentation techniques for autonomous navigation in space environments.
- Guo, Xianda et al. (2024): Proposed Stereo Anything, a unified framework for stereo matching using large-scale mixed data, enhancing depth estimation accuracy and robustness across diverse environments.

ROS2 & Drone Simulation Setup

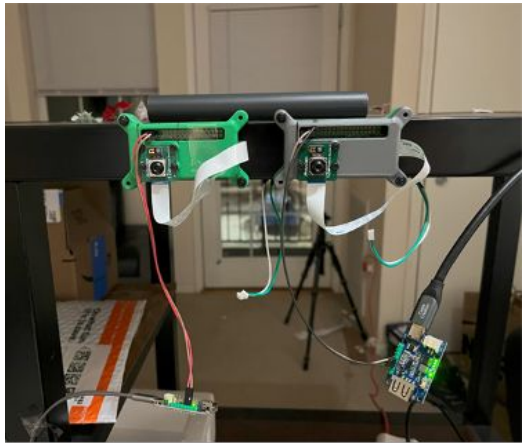
- ROS2-Humble Setup using Docker
- X11 forwarding
- Setup and Compile PX4 for SITL.
- Spin Up Gazebo and Isaac Sim for Simulation.
- Spin Up Micro-XRCE-DDS for PX4 to ROS2 communication.



ROS2 & Drone Simulation Setup

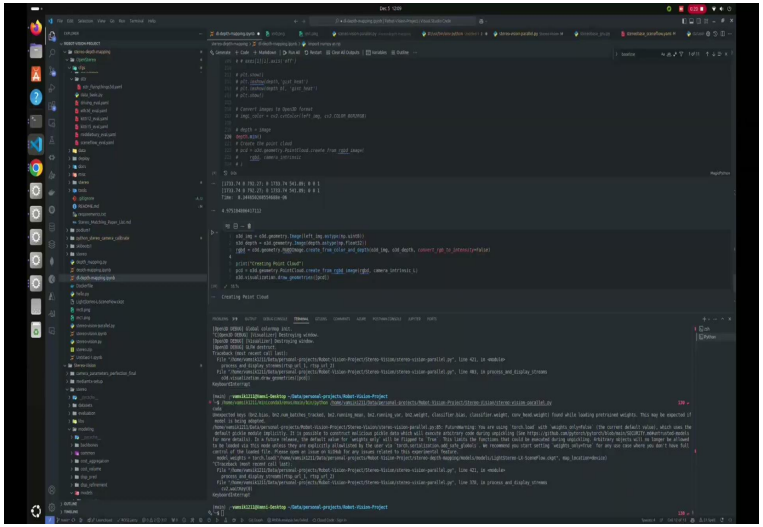
Point Cloud from Stereo Camera

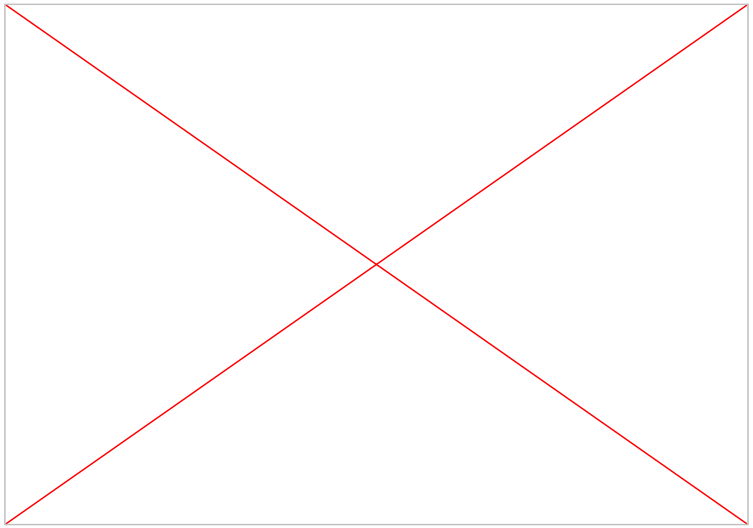
- 2 Raspi Zero with Camera Setup with SSH
- Configure video streaming
- Calibrate cameras for Intrinsic & Extrinsic Params.
- Receive video & process to generate disparity map.
- Create Point cloud from Depth Map using disparity map and camera intrinsic matrices.



Point Cloud from Stereo Camera

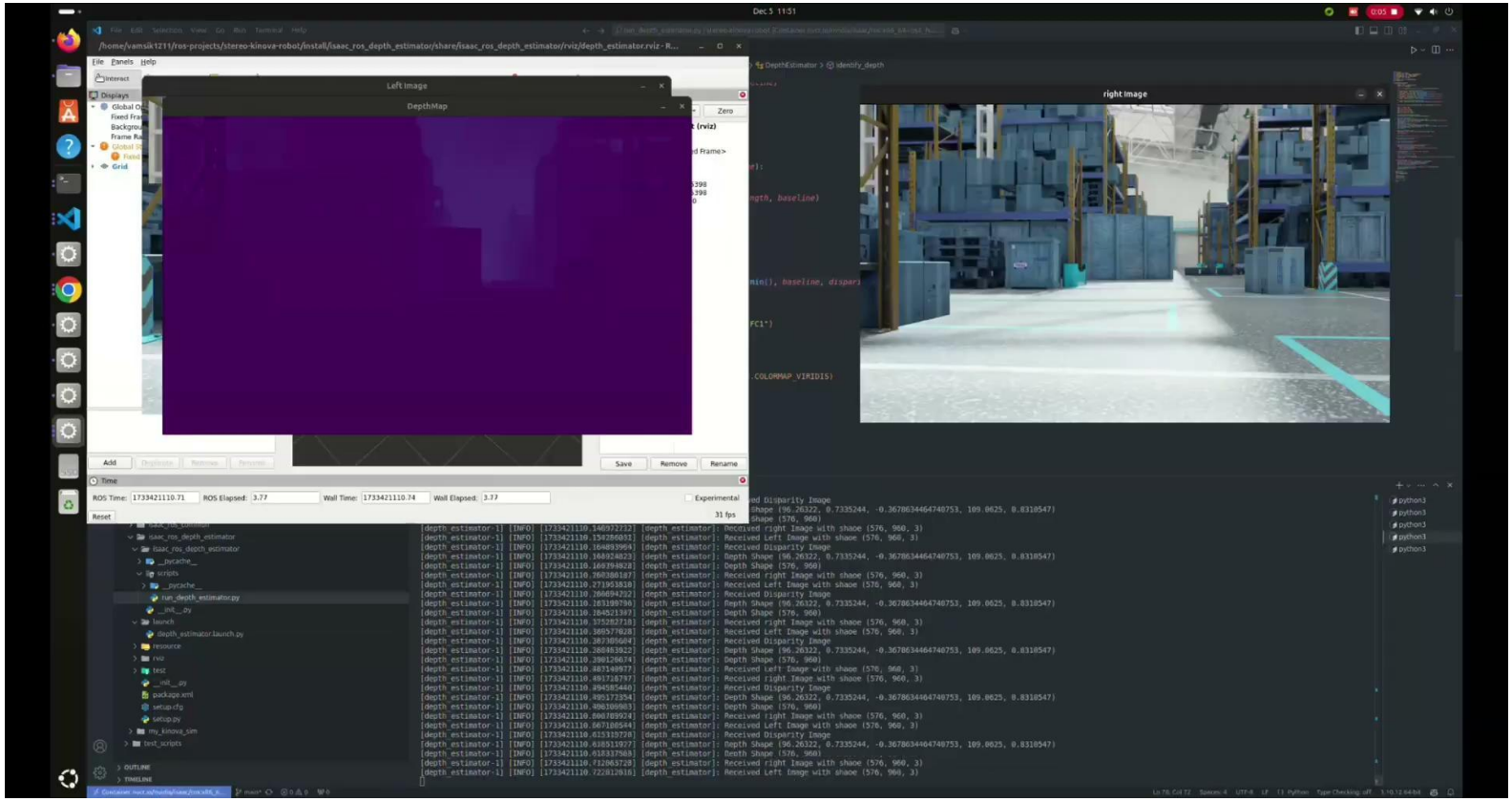
- Trained Light Stereo model (Using mobilenetV2)
- Tensorrt conversion for Real-Time inference.
- Inference below 2ms.





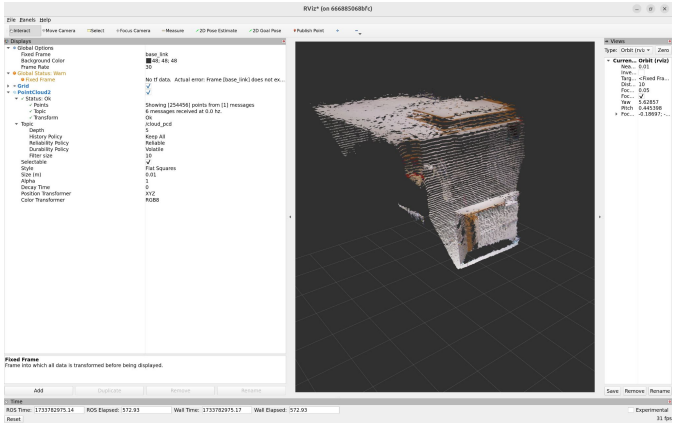
Depth Map in Real-Time

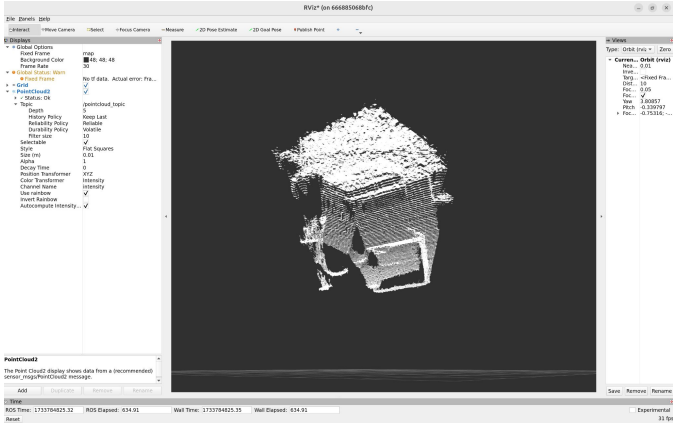
- Isaac ROS - Depth Mapping Node



Results: 3D Mapping from point cloud

- Point Cloud Library for pcl file: https://github.com/PointCloudLibrary/pcl/tree/master/test
- Published PointCloud2 topic using sample pcl file
- Subscribed to point cloud topic
  - NumPy and Open3D to visualize data





Publisher:

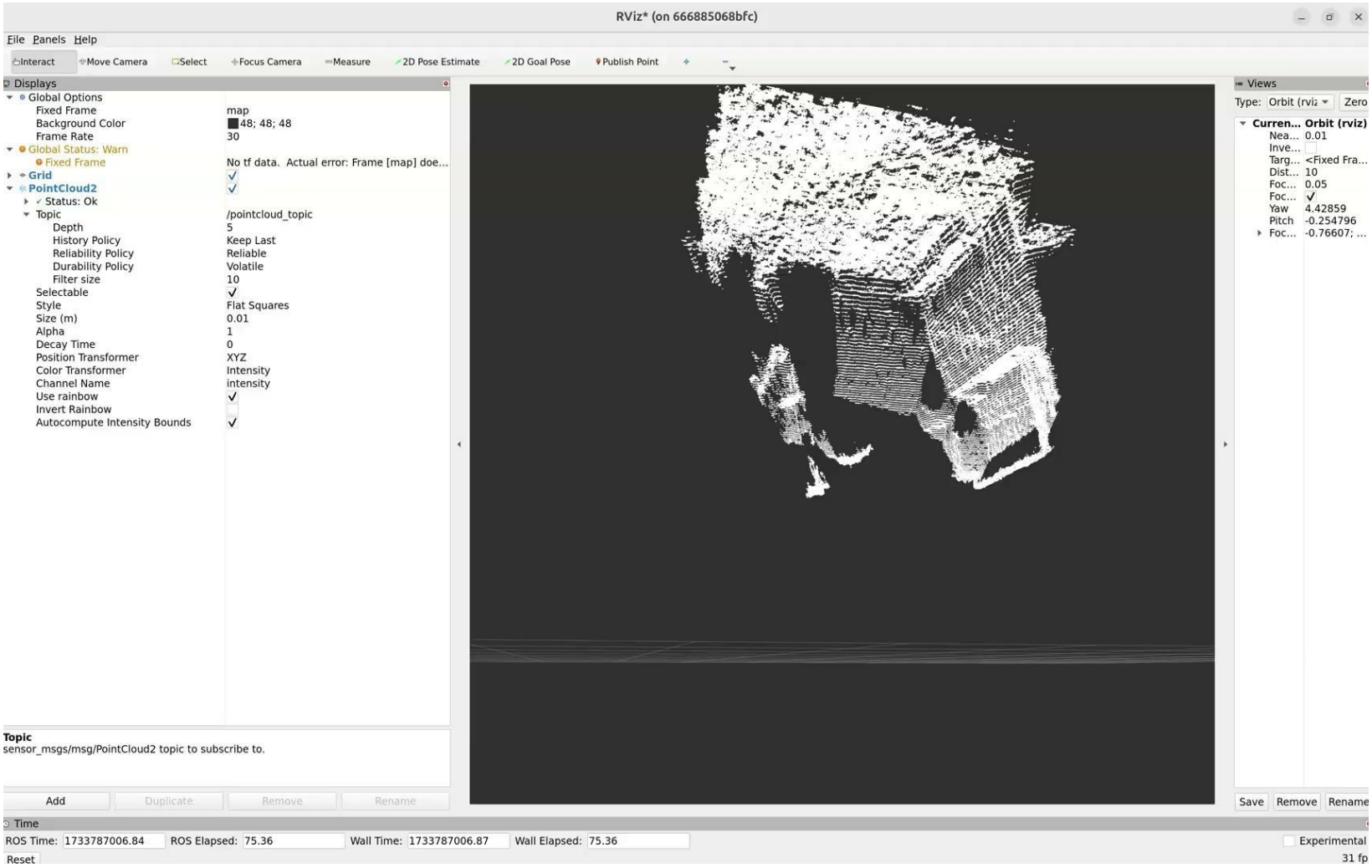
```
self.publisher = self.create_publisher(PointCloud2, 'pointcloud_topic', 10)
self.timer = self.create_timer(1.0, self.publish_pointcloud)  # Publish every 1 second
self.pcd_file = "/root/ros2_ws/src/car6.pcd"
```

Subscriber:

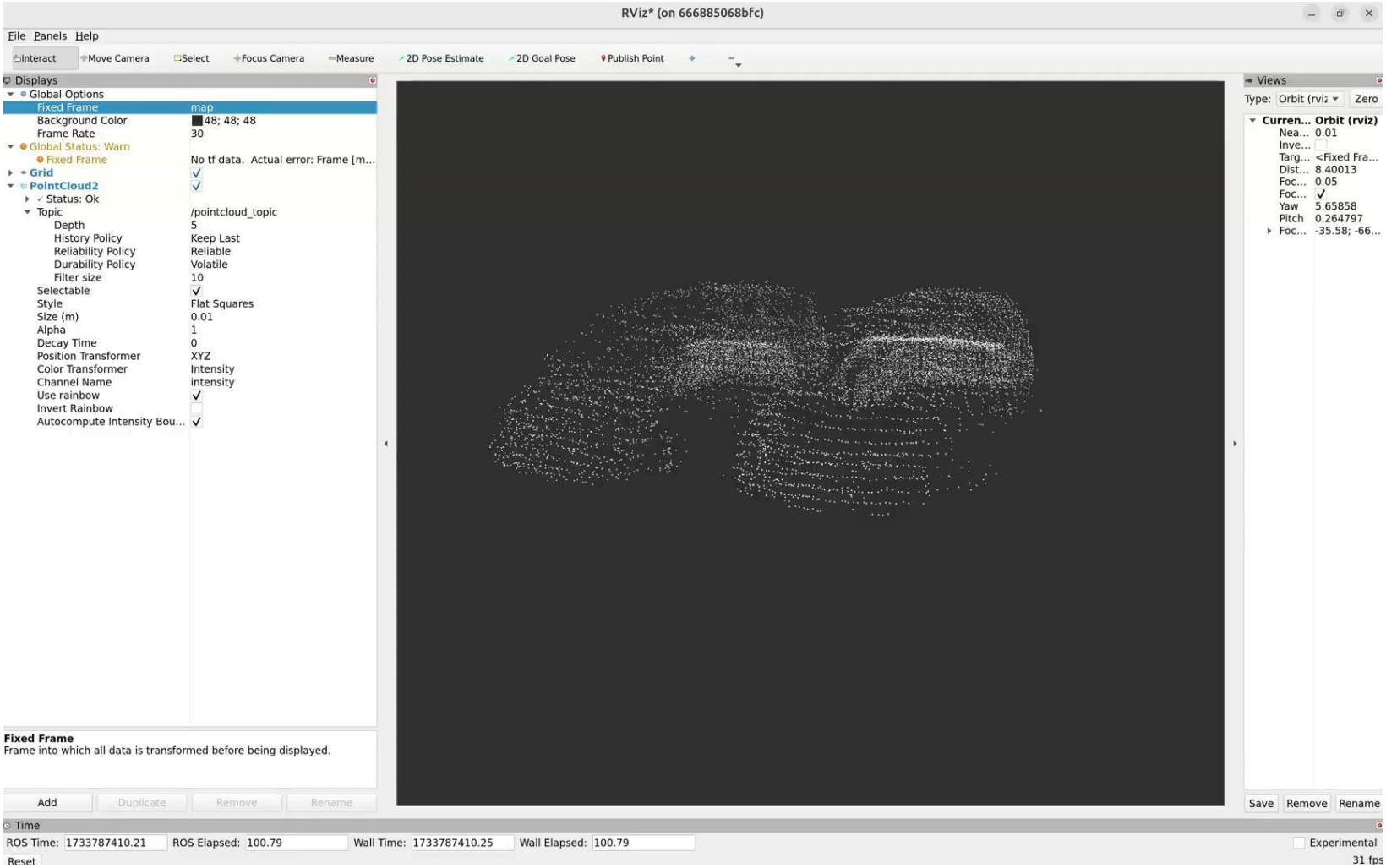
```
self.subscription = self.create_subscription(
    PointCloud2,
    'pointcloud_topic',  # Topic name from publisher
    self.callback,
    10  # QoS profile; stores 10 messages in history buffer
)
```

```
# Change PointCloud2 message into structured array
points_structured = np.array(
    list(point_cloud2.read_points(msg, field_names=("x", "y", "z"), skip_nans=True))
)

# Change structured array to NumPy array (x, y, z)
points = np.stack([points_structured['x'], points_structured['y'], points_structured['z']], axis=-1)

# Change to point cloud
cloud = o3d.geometry.PointCloud()
cloud.points = o3d.utility.Vector3dVector(points)

# Add to global point cloud
self.global_cloud += cloud
self.get_logger().info(f"Accumulated {len(self.global_cloud.points)} points.")

# Save the map as PCD file repeatedly
o3d.io.write_point_cloud("global_map.pcd", self.global_cloud)

# Shows the map in real time
o3d.visualization.draw_geometries([self.global_cloud])
```

Results: 3D Mapping from point cloud



Results: 3D Mapping from point cloud



Problems Encountered

Technical Issues:

- Stereo depth camera's disparity noise in high-reflectivity & low lit environments.
- Dependency issues, and needed to create multiple versions of docker images.

Simulation Bottlenecks:

- Gazebo rendering speed limited by GPU power.
- Isaac Sim & Isaac ROS hogging GPU Memory and processing.
- Distributed Compute setup up limitation due to Network bandwidth.

.

Future Work

- Integration of Stereo Vision and PX4
  - Combine stereo camera outputs with PX4 flight control for enhanced mapping and precise navigation.
- Distributed System Architecture
  - Enable ROS nodes to run across multiple machines, optimizing computational efficiency and ensuring robust real-time performance.
- Real-Time System Optimization
  - Reduce latency in data processing and synchronization for seamless drone operation environments.